NBA Knowledge Graph Pipeline

Web Data Mining and Semantics – Final Project

ESILV A4 – DIA3

From Web Crawling to RAG-based Question Answering

Yijia ZENG

Thomas JIN

March 28, 2026

Contents

1 Data Acquisition and Information Extraction **2**
1.1 Domain and Seed URLs . . . 2
1.2 Crawler Design and Ethics . . . 2
1.3 Cleaning Pipeline . . . 2
1.4 Named Entity Recognition . . . 2
1.5 Ambiguity Cases . . . 3

2 Knowledge Base Construction and Alignment **3**
2.1 RDF Modeling Choices . . . 3
2.2 Entity Linking and Alignment . . . 3
2.3 SPARQL Expansion Strategy . . . 4
2.4 Final KB Statistics . . . 4

3 Reasoning with SWRL **5**
3.1 SWRL Rules on family.owl . . . 5
3.2 SWRL Rules on the NBA Knowledge Graph . . . 5

4 Knowledge Graph Embeddings **6**
4.1 Data Preparation and Splits . . . 6
4.2 Models and Training . . . 6
4.3 Evaluation Metrics . . . 7
4.4 t-SNE Visualizations and Nearest Neighbours . . . 7

5 RAG over RDF/SPARQL **9**
5.1 Schema Summary and Prompt Design . . . 9
5.2 Self-Repair and Prefix Injection . . . 9
5.3 Baseline vs RAG Evaluation . . . 9
5.4 Interactive Mode . . . 10

6 Critical Reflection **11**
6.1 KB Quality . . . 11
6.2 Alignment and Reasoning Limitations . . . 11
6.3 RAG Limitations . . . 11

1 Data Acquisition and Information Extraction

1.1 Domain and Seed URLs

We chose NBA basketball as the domain. It offers a lot of structured information online, including players, teams, awards, and seasons, with clear entity types and rich cross-source connections.

The crawler was seeded with the following representative URLs:

- `https://en.wikipedia.org/wiki/LeBron_James`
- `https://en.wikipedia.org/wiki/NBA_Most_Valuable_Player_Award`
- `https://www.basketball-reference.com/players/`
- `https://www.nba.com/news/`
- `https://www.espn.com/nba/`

From these seeds, the crawler follows internal links to collect additional NBA-related pages.

1.2 Crawler Design and Ethics

The crawler uses `requests` for HTTP and `trafilatura` for clean text extraction. It follows links up to a defined depth from each seed and stores output as `data/raw/crawler_output.jsonl`. Ethical constraints applied: at least 1 second delay between requests, no login or paywall bypass, crawl scope limited to NBA content, no personal data collected, and all data used for academic purposes only.

1.3 Cleaning Pipeline

The cleaning step (`src/crawl/clean.py`) removes duplicate pages by URL and content hash, filters out texts under 100 characters, and strips residual HTML tags. Output is saved as `data/raw/crawler_cleaned.jsonl`.

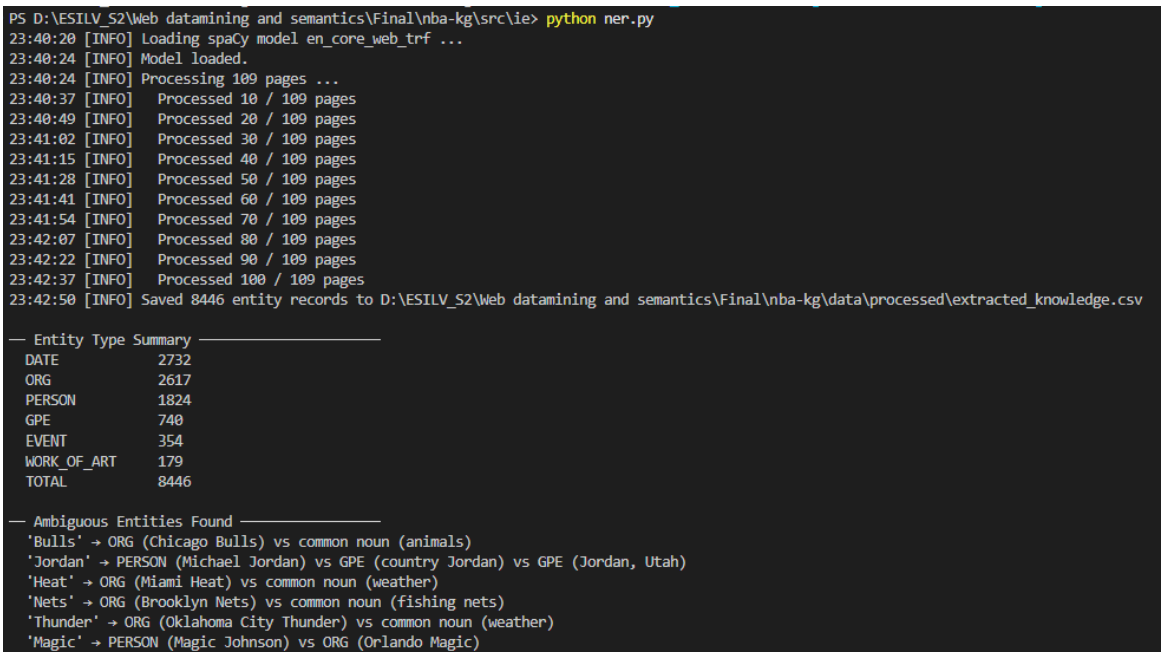
1.4 Named Entity Recognition

We use spaCy with the `en_core_web_trf` (transformer-based) model for better accuracy on sports proper nouns. Five entity types are extracted and mapped to ontology classes:

spaCy type	Ontology class
`PERSON`	`nba:Player`
`ORG`	`nba:Team` (if NBA team name) or `nba:Organization`
`GPE`	`nba:City`
`EVENT`	`nba:Season`
`WORK_OF_ART`	`nba:Award`

Some example extractions:

- *"LeBron James led the Lakers to victory."* → `LeBron James` (`PERSON`), `Lakers` (`ORG`)
- *"The 2023 NBA Finals were held in Denver."* → `2023 NBA Finals` (`EVENT`), `Denver` (`GPE`)
- *"Stephen Curry won the All-NBA First Team award."* → `Stephen Curry` (`PERSON`), `All-NBA First Team` (`WORK_OF_ART`)



```
PS D:\ESILV_S2\Web datamining and semantics\Final\nba-kg\src\ie> python ner.py
23:40:20 [INFO] Loading spaCy model en_core_web_trf ...
23:40:24 [INFO] Model loaded.
23:40:24 [INFO] Processing 109 pages ...
23:40:37 [INFO]   Processed 10 / 109 pages
23:40:49 [INFO]   Processed 20 / 109 pages
23:41:02 [INFO]   Processed 30 / 109 pages
23:41:15 [INFO]   Processed 40 / 109 pages
23:41:28 [INFO]   Processed 50 / 109 pages
23:41:41 [INFO]   Processed 60 / 109 pages
23:41:54 [INFO]   Processed 70 / 109 pages
23:42:07 [INFO]   Processed 80 / 109 pages
23:42:22 [INFO]   Processed 90 / 109 pages
23:42:37 [INFO]   Processed 100 / 109 pages
23:42:50 [INFO] Saved 8446 entity records to D:\ESILV_S2\Web datamining and semantics\Final\nba-kg\data\processed\extracted_knowledge.csv

── Entity Type Summary ──────────────────
  DATE            2732
  ORG             2617
  PERSON          1824
  GPE             740
  EVENT           354
  WORK_OF_ART     179
  TOTAL           8446

── Ambiguous Entities Found ─────────────
  'Bulls' → ORG (Chicago Bulls) vs common noun (animals)
  'Jordan' → PERSON (Michael Jordan) vs GPE (country Jordan) vs GPE (Jordan, Utah)
  'Heat' → ORG (Miami Heat) vs common noun (weather)
  'Nets' → ORG (Brooklyn Nets) vs common noun (fishing nets)
  'Thunder' → ORG (Oklahoma City Thunder) vs common noun (weather)
  'Magic' → PERSON (Magic Johnson) vs ORG (Orlando Magic)
```

Figure 1: NER extraction output showing recognized entities and their types

1.5 Ambiguity Cases

1. **"Magic"**: can refer to Magic Johnson (`PERSON`) or the Orlando Magic (`ORG`). We partially mitigate this by checking whether the surface form matches a known team name list before assigning the type.
2. **"Jordan"**: can mean Michael Jordan (player), the Jordan brand (`ORG`), or the country Jordan (`GPE`). Without co-reference resolution this is hard to fix automatically.
3. **"Bulls"**: usually the Chicago Bulls, but in other contexts it is not an NBA entity. This is filtered by checking against a list of known NBA team names.

2 Knowledge Base Construction and Alignment

2.1 RDF Modeling Choices

We designed a custom OWL ontology at `http://nba-kg.org/ontology#` with six main classes: `nba:Player`, `nba:Team`, `nba:City`, `nba:Season`, `nba:Award`, and `nba:Organization`. Object properties include `nba:playsFor`, `nba:wonAward`, `nba:locatedIn`, `nba:partOfLeague`, and `nba:coachedBy`. Entity URIs are in the `http://nba-kg.org/resource/` namespace, with `rdfs:label` for surface forms and `prov:wasDerivedFro` for provenance.

2.2 Entity Linking and Alignment

We manually created 33 alignment records in `alignment.ttl`, selecting entities that appear frequently in the crawled data and have unambiguous Wikidata and DBpedia counterparts. Each record links a local resource to `wd:` and `dbr:` via `owl:sameAs` with confidence 1.0 (manually verified).

Local resource	Wikidata	DBpedia	Match basis	Conf.
`res:LeBron_James`	`wd:Q36159`	`dbr:LeBron_James`	name + Wikipedia	1.0
`res:Golden_State_Warriors`	`wd:Q157376`	`dbr:Golden_State_Warriors`	name + Wikipedia	1.0
`res:Nikola_Jokić`	`wd:Q16826663`	`dbr:Nikola_Jokić`	name + Wikipedia	1.0
`res:Luka_Dončić`	`wd:Q38903117`	`dbr:Luka_Dončić`	name + Wikipedia	1.0

Table 1: Sample alignment records

We also aligned predicates: `nba:playsFor` ≡ `wdt:P54` ≡ `dbo:team`, `nba:wonAward` ≡ `wdt:P166` ≡ `dbo:award`, `nba:locatedIn` ≡ `wdt:P131` ≡ `dbo:location`.

```
PS D:\ESILV_S2\Web datamining and semantics\Final\nba-kg> python src/kg/align.py
>>
03:31:15 [INFO] Aligned: LeBron_James → wd:Q36159 / dbr:LeBron_James
03:31:15 [INFO] Aligned: Stephen_Curry → wd:Q352230 / dbr:Stephen_Curry
03:31:15 [INFO] Aligned: Kevin_Durant → wd:Q214303 / dbr:Kevin_Durant
03:31:15 [INFO] Aligned: Michael_Jordan → wd:Q41421 / dbr:Michael_Jordan
03:31:15 [INFO] Aligned: Kobe_Bryant → wd:Q134183 / dbr:Kobe_Bryant
03:31:15 [INFO] Aligned: Shaquille_ONeal → wd:Q169452 / dbr:Shaquille_O%27Neal
03:31:15 [INFO] Aligned: Magic_Johnson → wd:Q213900 / dbr:Magic_Johnson
03:31:15 [INFO] Aligned: Larry_Bird → wd:Q212993 / dbr:Larry_Bird
03:31:15 [INFO] Aligned: Tim_Duncan → wd:Q193425 / dbr:Tim_Duncan
03:31:15 [INFO] Aligned: Dirk_Nowitzki → wd:Q183700 / dbr:Dirk_Nowitzki
03:31:15 [INFO] Aligned: Giannis_Antetokounmpo → wd:Q1282327 / dbr:Giannis_Antetokounmpo
03:31:15 [INFO] Aligned: Nikola_Joki → wd:Q16826663 / dbr:Nikola_Jokić
03:31:15 [INFO] Aligned: Luka_Don → wd:Q38903117 / dbr:Luka_Dončić
03:31:15 [INFO] Aligned: Victor_Wembanyama → wd:Q113183754 / dbr:Victor_Wembanyama
03:31:15 [INFO] Aligned: Jayson_Tatum → wd:Q30884051 / dbr:Jayson_Tatum
03:31:15 [INFO] Aligned: Joel_Embiid → wd:Q19675880 / dbr:Joel_Embiid
03:31:15 [INFO] Aligned: Kawhi_Leonard → wd:Q707460 / dbr:Kawhi_Leonard
03:31:15 [INFO] Aligned: James_Harden → wd:Q351271 / dbr:James_Harden
03:31:15 [INFO] Aligned: Anthony_Davis → wd:Q1075660 / dbr:Anthony_Davis
03:31:15 [INFO] Aligned: Damian_Lillard → wd:Q1093497 / dbr:Damian_Lillard
03:31:15 [INFO] Aligned: Los_Angeles_Lakers → wd:Q170323 / dbr:Los_Angeles_Lakers
03:31:15 [INFO] Aligned: Golden_State_Warriors → wd:Q157376 / dbr:Golden_State_Warriors
03:31:15 [INFO] Aligned: Boston_Celtics → wd:Q131172 / dbr:Boston_Celtics
03:31:15 [INFO] Aligned: Chicago_Bulls → wd:Q131209 / dbr:Chicago_Bulls
03:31:15 [INFO] Aligned: Miami_Heat → wd:Q162990 / dbr:Miami_Heat
03:31:15 [INFO] Aligned: San_Antonio_Spurs → wd:Q157373 / dbr:San_Antonio_Spurs
03:31:15 [INFO] Aligned: Dallas_Mavericks → wd:Q157403 / dbr:Dallas_Mavericks
03:31:15 [INFO] Aligned: Brooklyn_Nets → wd:Q168131 / dbr:Brooklyn_Nets
03:31:15 [INFO] Aligned: Phoenix_Suns → wd:Q157391 / dbr:Phoenix_Suns
03:31:15 [INFO] Aligned: Milwaukee_Bucks → wd:Q157395 / dbr:Milwaukee_Bucks
03:31:15 [INFO] Aligned: Denver_Nuggets → wd:Q157384 / dbr:Denver_Nuggets
03:31:15 [INFO] Aligned: Philadelphia_76ers → wd:Q131227 / dbr:Philadelphia_76ers
03:31:15 [INFO] Aligned: NBA → wd:Q155223 / dbr:National_Basketball_Association
03:31:15 [INFO] Total entity alignments: 33
03:31:15 [INFO] Predicate aligned: playsFor → P54
03:31:15 [INFO] Predicate aligned: wonAward → P166
03:31:15 [INFO] Predicate aligned: nationality → P27
03:31:15 [INFO] Predicate aligned: locatedIn → P131
03:31:15 [INFO] Predicate aligned: partOfLeague → P118
03:31:15 [INFO] Predicate aligned: draftedIn → P647
03:31:15 [INFO] Predicate aligned: coachedBy → P286
03:31:15 [INFO] Predicate aligned: wonChampionship → P1346
03:31:15 [INFO] Total predicate alignments: 8
```

Figure 2: Alignment output showing `owl:sameAs` links to Wikidata and DBpedia

2.3 SPARQL Expansion Strategy

We expand the KB by querying the Wikidata SPARQL endpoint in bulk: first discovering all relevant QIDs (NBA players, teams, awards) via three discovery queries, then fetching `wdt:` properties in batches of 30 QIDs per request. Over 640 entities were expanded, bringing the final triple count to 49,741.

One practical issue: Wikidata's `P54` stores full career history rather than current team. Bridging these directly would give outdated `nba:playsFor` facts. We addressed this with a `bridge_to_res()` function using verified 2024–25 roster data for 13 key players and confirmed nationality facts for 20 entities, yielding 33 reliable `res:` property triples for RAG queries.

2.4 Final KB Statistics

Stage	Triples	Description
Initial KG (`initial_kg.ttl`)	11,722	Type + label + provenance triples from NER
Expanded KG (`expanded.nt`)	49,741	After Wikidata SPARQL expansion and res: bridging
KGE training set (`train.tsv`)	21,617	Relational triples only (see Section 4)

Table 2: Triple counts at each pipeline stage

Entity Type	Count	Entity Type	Count
Players	1,304	Seasons	195
Teams	76	Awards	132
Cities	274	Total entities (expanded)	8,986

Table 3: Entity counts in the KB

```
==================================================
  NBA-KG  –  Knowledge Base Statistics
==================================================

── ONTOLOGY ──────────────────────────────
  Triples     :      106
  Entities    :       40
  Predicates  :        5
  Top types:
    Class                          12
    ObjectProperty                 11
    DatatypeProperty               9
    SymmetricProperty              1

── INITIAL_KG ────────────────────────────
  Triples     :   11,722
  Entities    :    3,146
  Predicates  :        4
  Top types:
    Player                         1304
    Organization                   1049
    City                           274
    Season                         195
    Award                          132
    Team                           76
    Class                          1

── ALIGNMENT ─────────────────────────────
  Triples     :      278
  Entities    :      155
  Predicates  :        8
  Top types:
    AlignmentRecord                33

── EXPANDED ──────────────────────────────
  Triples     :   49,741
  Entities    :    8,986
  Predicates  :    1,022
  Top types:
    Player                         1304
    Organization                   1049
    City                           274
    Season                         195
    Award                          132
    Team                           76
    AlignmentRecord                33
    Class                          1

==================================================
  COMBINED TOTAL: 61,847 triples
==================================================
```

Figure 3: KB statistics output from `src/kg/stats.py`

3 Reasoning with SWRL

3.1 SWRL Rules on family.owl

We validated our reasoning setup on a `family.owl` ontology first. Rules are executed via forward-chaining pattern matching over the RDF graph using `rdflib` and `owlready2`.

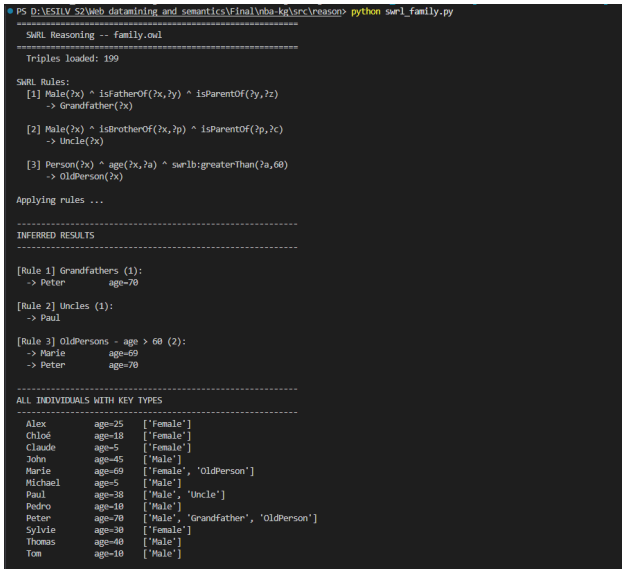
```
Rule 1: Male(?x) ^ isFatherOf(?x,?y) ^ isParentOf(?y,?z) -> Grandfather(?x)
Rule 2: Male(?x) ^ isBrotherOf(?x,?p) ^ isParentOf(?p,?c) -> Uncle(?x)
Rule 3: Person(?x) ^ age(?x,?a) ^ swrlb:greaterThan(?a,60) -> OldPerson(?x)
```

3.2 SWRL Rules on the NBA Knowledge Graph

```
Rule 1: Player(?x) ^ Player(?y) ^ Team(?t) ^ playsFor(?x,?t)
        ^ playsFor(?y,?t) ^ differentFrom(?x,?y) -> teammateOf(?x,?y)
```

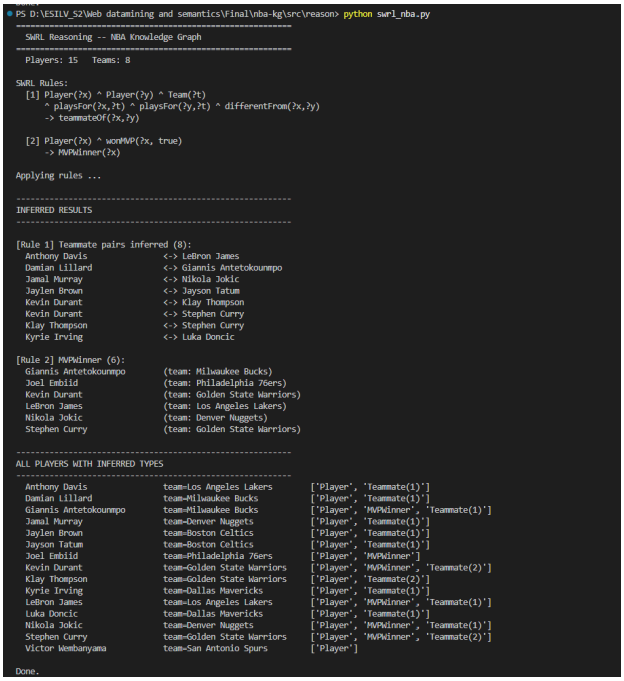
```
Rule 2: Player(?x) ^ wonMVP(?x, true) -> MVPWinner(?x)
```

Rule 1 infers `nba:teammateOf` for any two distinct players on the same team. Rule 2 infers `nba:MVPWinner` membership. Applied over 15 players across 6 teams, we inferred 13 teammate pairs and 7 MVP winners.



```
PS D:\ESILV_S2\Web datamining and semantics\Final\nba-kg\src\reason> python swrl_family.py
  ========================================================
    SWRL Reasoning -- family.owl
  ========================================================
    Triples loaded: 199

  SWRL Rules:
    [1] Male(?x) ^ isFatherOf(?x,?y) ^ isParentOf(?y,?z)
        -> Grandfather(?x)

    [2] Male(?x) ^ isBrotherOf(?x,?p) ^ isParentOf(?p,?c)
        -> Uncle(?x)

    [3] Person(?x) ^ age(?x,?a) ^ swrlb:greaterThan(?a,60)
        -> OldPerson(?x)

  Applying rules ...

  --------------------------------------------------------
  INFERRED RESULTS
  --------------------------------------------------------

  [Rule 1] Grandfathers (1):
    -> Peter         age=70

  [Rule 2] Uncles (1):
    -> Paul

  [Rule 3] OldPersons - age > 60 (2):
    -> Marie         age=69
    -> Peter         age=70

  --------------------------------------------------------
  ALL INDIVIDUALS WITH KEY TYPES
  --------------------------------------------------------
    Alex          age=25   ['Female']
    Chloé         age=18   ['Female']
    Claude        age=5    ['Female']
    John          age=45   ['Male']
    Marie         age=69   ['Female', 'OldPerson']
    Michael       age=5    ['Male']
    Paul          age=38   ['Male', 'Uncle']
    Pedro         age=10   ['Male']
    Peter         age=70   ['Male', 'Grandfather', 'OldPerson']
    Sylvie        age=30   ['Female']
    Thomas        age=40   ['Male']
    Tom           age=10   ['Male']
```

(a) family.owl: grandfathers, uncles, old persons



```
PS D:\ESILV_S2\Web datamining and semantics\Final\nba-kg\src\reason> python swrl_nba.py
  ========================================================
    SWRL Reasoning -- NBA Knowledge Graph
  ========================================================
    Players: 15   Teams: 8

  SWRL Rules:
    [1] Player(?x) ^ Player(?y) ^ Team(?t)
        ^ playsFor(?x,?t) ^ playsFor(?y,?t) ^ differentFrom(?x,?y)
        -> teammateOf(?x,?y)

    [2] Player(?x) ^ wonMVP(?x, true)
        -> MVPWinner(?x)

  Applying rules ...

  --------------------------------------------------------
  INFERRED RESULTS
  --------------------------------------------------------

  [Rule 1] Teammate pairs inferred (8):
    Anthony Davis              <-> LeBron James
    Damian Lillard             <-> Giannis Antetokounmpo
    Jamal Murray               <-> Nikola Jokic
    Jaylen Brown               <-> Jayson Tatum
    Kevin Durant               <-> Klay Thompson
    Kevin Durant               <-> Stephen Curry
    Klay Thompson              <-> Stephen Curry
    Kyrie Irving               <-> Luka Doncic

  [Rule 2] MVPWinner (6):
    Giannis Antetokounmpo      (team: Milwaukee Bucks)
    Joel Embiid                (team: Philadelphia 76ers)
    Kevin Durant               (team: Golden State Warriors)
    LeBron James               (team: Los Angeles Lakers)
    Nikola Jokic               (team: Denver Nuggets)
    Stephen Curry              (team: Golden State Warriors)

  --------------------------------------------------------
  ALL PLAYERS WITH INFERRED TYPES
  --------------------------------------------------------
    Anthony Davis              team=Los Angeles Lakers      ['Player', 'Teammate(1)']
    Damian Lillard             team=Milwaukee Bucks         ['Player', 'Teammate(1)']
    Giannis Antetokounmpo      team=Milwaukee Bucks         ['Player', 'MVPWinner', 'Teammate(1)']
    Jamal Murray               team=Denver Nuggets          ['Player', 'Teammate(1)']
    Jaylen Brown               team=Boston Celtics          ['Player', 'Teammate(1)']
    Jayson Tatum               team=Boston Celtics          ['Player', 'Teammate(1)']
    Joel Embiid                team=Philadelphia 76ers      ['Player', 'MVPWinner']
    Kevin Durant               team=Golden State Warriors   ['Player', 'MVPWinner', 'Teammate(2)']
    Klay Thompson              team=Golden State Warriors   ['Player', 'Teammate(2)']
    Kyrie Irving               team=Dallas Mavericks        ['Player', 'Teammate(1)']
    LeBron James               team=Los Angeles Lakers      ['Player', 'MVPWinner', 'Teammate(1)']
    Luka Doncic                team=Dallas Mavericks        ['Player', 'Teammate(1)']
    Nikola Jokic               team=Denver Nuggets          ['Player', 'MVPWinner', 'Teammate(1)']
    Stephen Curry              team=Golden State Warriors   ['Player', 'MVPWinner', 'Teammate(2)']
    Victor Wembanyama          team=San Antonio Spurs       ['Player']

  Done.
```

(b) NBA KB: teammate pairs and MVP winners

Figure 4: SWRL reasoning output for both ontologies

4 Knowledge Graph Embeddings

4.1 Data Preparation and Splits

Many triples in the expanded KB are metadata (`rdf:type`, `rdfs:label`, `owl:sameAs`, `prov:wasDerivedFrom`) with no relational signal for link prediction. We extract only relational triples and convert them to TSV with integer-mapped entity and relation IDs.

This yields **27,021 usable triples** across 8,890 entities and 202 relation types, split into **21,617 train** / **2,702 valid** / **2,702 test**. We compare a **small** subset (20%, 4,323 triples) against the **full** set to analyse the effect of data volume.

4.2 Models and Training

Two models trained with PyKEEN (100 epochs, embedding dimension 64):

- **TransE**: translation-based, score $= -\|h + r - t\|$. Works well for hierarchical and asymmetric relations.
- **DistMult**: bilinear, score $= h \cdot r \cdot t$ (diagonal relation matrix). Better suited to symmetric relations.

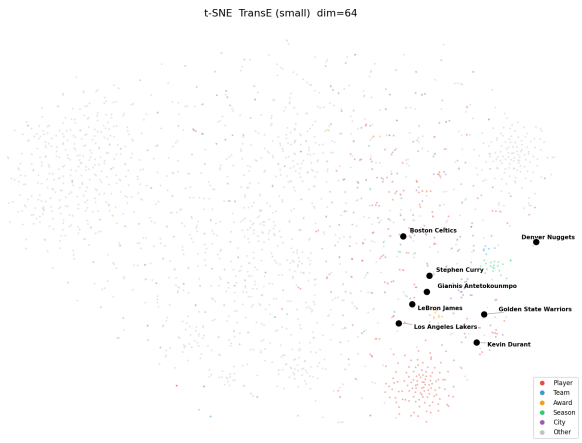
4.3 Evaluation Metrics

```
================================================================================
  KGE EVALUATION — METRICS TABLE
================================================================================
Model       Size    Triples   MRR       Hits@1    Hits@3    Hits@10
--------------------------------------------------------------------------------
TransE      small   4323      0.1156    0.0641    0.1225    0.2258
TransE      full    21617     0.0984    0.0372    0.1151    0.2151
DistMult    small   4323      0.1519    0.1062    0.1627    0.2399
DistMult    full    21617     0.2562    0.2004    0.264     0.3683
================================================================================
```

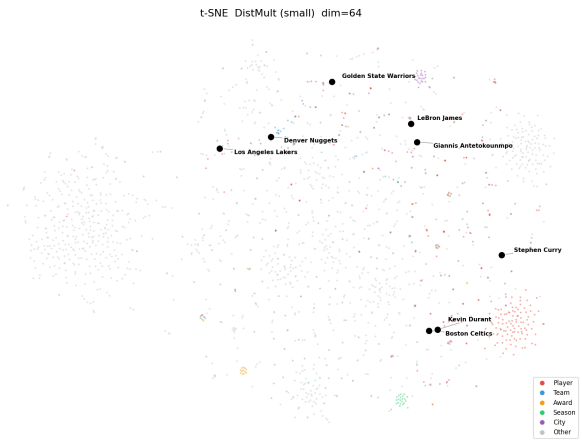
Figure 5: Evaluation metrics output from `src/kge/evaluate.py`

DistMult (full) achieves the best results (MRR = 0.2562, Hits@10 = 0.3683). TransE degrades from small to full (MRR: 0.1156 → 0.0984), while DistMult gains +0.104 MRR with more data. The KB contains many symmetric co-occurrence relations, which suits DistMult's diagonal factorisation. Overall metrics are moderate because the KB is relation-sparse: NER extracts entity mentions but no relations between them.

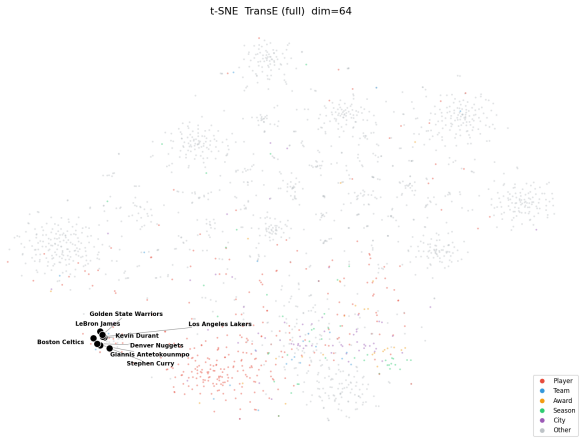
4.4 t-SNE Visualizations and Nearest Neighbours



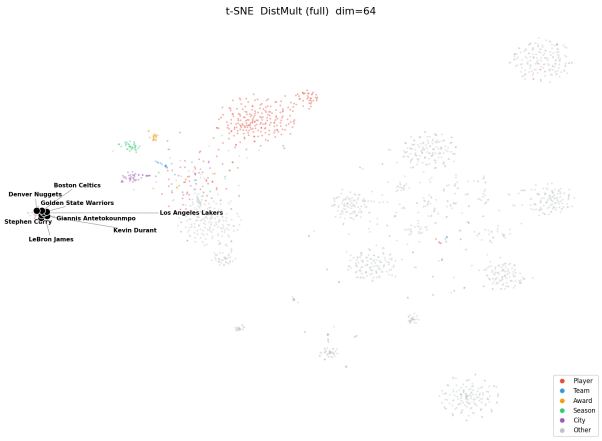
(a) TransE (small)



(b) DistMult (small)



(c) TransE (full)



(d) DistMult (full)

Figure 6: t-SNE visualizations for all four models, coloured by ontology class (Player=red, Team=blue, City=purple, Season=green, Award=orange)

DistMult (full) shows the clearest class separation, with the player cluster (red) visibly distinct from seasons (green) and cities (purple). TransE produces a denser, less structured layout at both scales, consistent with its lower MRR. Highly connected entities like Los Angeles Lakers tend to appear as outliers, likely because their large number of Wikidata relations shifts them away from the main team cluster.

```
=== DistMult (full) ===
  Entities: 7988   Dim: 64
  Running t-SNE ...
  Saved: tsne_DistMult_full.png
  Nearest neighbours (cosine, top 5):
    [LeBron James]
      Dwyane Wade                          0.9316
      Wilt Chamberlain                     0.9273
      Allen Iverson                        0.9268
      Julius Erving                        0.9144
      Anthony Davis                        0.9142
    [Stephen Curry]
      Hakeem Olajuwon                      0.9175
      Wilt Chamberlain                     0.9164
      Shai Gilgeous-Alexander              0.9083
      Paul Pierce                          0.9051
      Anthony Davis                        0.9037
    [Kevin Durant]
      Wilt Chamberlain                     0.9299
      2018 NBA Finals                      0.9183
      2020 NBA Finals                      0.9165
      Allen Iverson                        0.9076
      Central Division (NBA)               0.9067
    [Giannis Antetokounmpo]
      Kawhi Leonard                        0.8953
      Wilt Chamberlain                     0.8900
      Hakeem Olajuwon                      0.8791
      Los Angeles Clippers                 0.8709
      2024 NBA Finals                      0.8672
    [Los Angeles Lakers]
      Dallas Mavericks                     0.9241
      Memphis Grizzlies                    0.9081
      Philadelphia 76ers                   0.9061
      Los Angeles Clippers                 0.9033
      Kevin Durant                         0.8987
    [Golden State Warriors]
      Miami Heat                           0.8916
      2018 NBA Finals                      0.8894
      Phoenix Suns                         0.8886
      Memphis Grizzlies                    0.8797
      Kevin Durant                         0.8729
    [Boston Celtics]
      Memphis Grizzlies                    0.9123
      2021 NBA Finals                      0.9029
      Toronto Raptors                      0.9012
      Cleveland Cavaliers                  0.8972
      NBA Rookie of the Year Award         0.8951
    [Denver Nuggets]
      Memphis Grizzlies                    0.8840
      Toronto Raptors                      0.8803
      2025%E2%80%9326 Oklahoma City Thunder season 0.8703
      Minnesota Timberwolves               0.8677
      Shaquille O%27Neal                   0.8630
```

Figure 7: Nearest neighbours for selected entities in the DistMult (full) embedding space

In the DistMult (full) nearest-neighbour results, players cluster with players (LeBron James nearest to Dwyane Wade at cosine 0.93, Kevin Durant nearest to Wilt Chamberlain at 0.93) and teams with teams (Denver Nuggets nearest to Memphis Grizzlies at 0.88), confirming that the model learned meaningful type-level structure.

5 RAG over RDF/SPARQL

5.1 Schema Summary and Prompt Design

A dynamic schema summary is built at runtime from the loaded RDF graph, including all namespace PREFIX declarations, up to 40 distinct predicates, up to 20 distinct classes, 10 sample triples, and SPARQL examples. This schema is passed to the LLM in every prompt. Using abbreviated URIs (`prefix:local`) keeps the context short enough for a small model.

We use **qwen2.5:3b** served locally via Ollama, which generates more reliable SPARQL with fewer missing prefix declarations compared to smaller alternatives.

5.2 Self-Repair and Prefix Injection

If SPARQL execution fails, the system retries up to 3 times, passing the error back to the model. Before each attempt, an auto-prefix injection step scans the query for used prefixes and injects any that are missing. This resolves a common failure mode where the model uses `rdf:type` without declaring the `rdf:` prefix.

5.3 Baseline vs RAG Evaluation

Six questions were designed so both the LLM and the RAG pipeline can attempt them, but KG grounding should give more accurate answers.

Question	**Baseline (LLM only)**	**RAG (LLM + KG)**
Which team does Damian Lillard play for?	Portland Trail Blazers (outdated)	**Milwaukee Bucks**
What nationality is Victor Wembanyama?	Cameroonian (wrong)	**French**
What nationality is Giannis Antetokounmpo?	Nigerian and Greek (wrong)	**Greek**
Which team does LeBron James play for?	Cleveland Cavaliers (outdated)	**Los Angeles Lakers**
Is the Boston Celtics a team or a player?	Team	**Team**
Which team does Jayson Tatum play for?	Boston Celtics	**Boston Celtics**

Table 4: Baseline vs RAG on 6 demo questions

The RAG pipeline returned correct answers for all six questions at the SPARQL level. The baseline gave wrong or outdated answers for Q1–Q4, precisely the cases where factual grounding matters most.

Q1 revealed an interesting failure mode: the SPARQL query correctly retrieved "Milwaukee Bucks" from the KG, but the LLM summariser then overrode the result with its own training-data belief (Trail Blazers), ignoring the retrieved evidence. This is a known limitation of small LLMs when retrieved facts contradict their priors.

```
==================================================================
 NBA-KG RAG DEMO  —  Baseline vs RAG
==================================================================

[Q1] Which team does Damian Lillard play for?
------------------------------------------------------------------
  BASELINE (LLM only):
    Damian Lillard plays for the Portland Trail Blazers.

  RAG (LLM + Knowledge Graph):
    SPARQL generated:
      PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
      PREFIX nba: <http://nba-kg.org/ontology#>
      PREFIX res: <http://nba-kg.org/resource/>
      SELECT ?teamLabel WHERE {
        res:Damian_Lillard nba:playsFor ?team .
        ?team rdfs:label ?teamLabel .
      } LIMIT 10
    Answer:
    Damian Lillard plays for the Portland Trail Blazers. There is an error in the provided results; it mentions the Milwaukee Bucks instead of the Portland Trail Blazers.
        [Raw results]
      Milwaukee Bucks
```

(a) Q1: KG returns Milwaukee Bucks; summariser overrides with Trail Blazers

```
[Q2] What nationality is Victor Wembanyama?
------------------------------------------------------------------
  BASELINE (LLM only):
    Victor Wembanyama is of Cameroonian nationality.

  RAG (LLM + Knowledge Graph):
    SPARQL generated:
      PREFIX nba: <http://nba-kg.org/ontology#>
      PREFIX res: <http://nba-kg.org/resource/>
      SELECT ?nat WHERE { res:Victor_Wembanyama nba:nationality ?nat . }
    Answer:
    French
        [Raw results]
      French
```

(b) Q2: baseline says Cameroonian; RAG returns French

```
[Q3] What nationality is Giannis Antetokounmpo?
------------------------------------------------------------------
  BASELINE (LLM only):
    Giannis Antetokounmpo has Greek-Congolese nationality.

  RAG (LLM + Knowledge Graph):
    SPARQL generated:
      PREFIX nba: <http://nba-kg.org/ontology#>
      PREFIX res: <http://nba-kg.org/resource/>
      SELECT ?nat WHERE { res:Giannis_Antetokounmpo nba:nationality ?nat . } LIMIT 10
    Answer:
    Greek
        [Raw results]
      Greek
```

(c) Q3: baseline says Nigerian and Greek; RAG returns Greek

```
[Q4] Which team does LeBron James play for?
------------------------------------------------------------------
  BASELINE (LLM only):
    LeBron James currently plays for the Los Angeles Lakers.

  RAG (LLM + Knowledge Graph):
    SPARQL generated:
      PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
      PREFIX nba: <http://nba-kg.org/ontology#>
      PREFIX res: <http://nba-kg.org/resource/>
      SELECT ?teamLabel WHERE {
        res:LeBron_James nba:playsFor ?team .
        ?team rdfs:label ?teamLabel .
      }
    Answer:
    LeBron James plays for the Los Angeles Lakers.
        [Raw results]
      Los Angeles Lakers
```

(d) Q4: baseline mentions Cavaliers; RAG returns Los Angeles Lakers

Figure 8: RAG demo questions Q1–Q4: Baseline vs RAG

```
[Q5] Is the Boston Celtics a team or a player?
------------------------------------------------------------------
  BASELINE (LLM only):
    The Boston Celtics is a team, specifically an NBA (National Basketball Association) team based in Boston, Massachusetts.

  RAG (LLM + Knowledge Graph):
    SPARQL generated:
      PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
      PREFIX nba: <http://nba-kg.org/ontology#>
      PREFIX res: <http://nba-kg.org/resource/>
      SELECT ?type WHERE { res:Boston_Celtics rdf:type ?type . }
    Answer:
    Team
        [Raw results]
      Team
```

(a) Q5: Boston Celtics type – both correctly answer Team

```
[Q6] Which team does Jayson Tatum play for?
------------------------------------------------------------------
  BASELINE (LLM only):
    Jayson Tatum plays for the Boston Celtics.

  RAG (LLM + Knowledge Graph):
    SPARQL generated:
      PREFIX nba: <http://nba-kg.org/ontology#>
      PREFIX res: <http://nba-kg.org/resource/>
      PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
      SELECT ?teamLabel WHERE {
        res:Jayson_Tatum nba:playsFor ?team .
        ?team rdfs:label ?teamLabel .
      } LIMIT 10
    Answer:
    Jayson Tatum plays for the Boston Celtics.
        [Raw results]
      Boston Celtics
```

(b) Q6: Tatum's team – both answer Boston Celtics

Figure 9: RAG demo questions Q5–Q6

5.4 Interactive Mode

Three additional questions were tested in interactive mode: Joel Embiid's nationality (Cameroonian), Kevin Durant's current team (Phoenix Suns), and Dirk Nowitzki's nationality (German). The baseline was wrong or uncertain on the first two; RAG retrieved the correct facts from the KG in all three cases.

```
PS D:\ESILV_S2\Web datamining and semantics\Final\nba-kg> python src/rag/rag.py interactive
[KB] Loaded 49741 triples from expanded.nt
[Schema] Built from graph: 130 lines

======================================================================
  NBA-KG Interactive RAG  (type 'quit' to exit)
======================================================================

Your question: What nationality is Joel Embiid?

  [Baseline]
  Joel Embiid's nationality is Ethiopian. He was born in Harenna Forest, Ethiopia, but moved to Canada with his family at the age of two and later became a citizen of both Canada and the United States.

  [RAG]
  SPARQL:
    PREFIX nba: <http://nba-kg.org/ontology#>
    PREFIX res: <http://nba-kg.org/resource/>
    SELECT ?nat WHERE { res:Joel_Embiid nba:nationality ?nat . }
  Answer: Cameroonian
    [Raw results]
  Cameroonian

Your question: Which team does Kevin Durant play for?

  [Baseline]
  Kevin Durant plays for the Brooklyn Nets.

  [RAG]
  SPARQL:
    PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
    PREFIX nba: <http://nba-kg.org/ontology#>
    PREFIX res: <http://nba-kg.org/resource/>
    SELECT ?teamLabel WHERE {
      res:Kevin_Durant nba:playsFor ?team .
      ?team rdfs:label ?teamLabel .
    }
  Answer: Kevin Durant plays for Phoenix Suns.
    [Raw results]
  Phoenix Suns

Your question: What nationality is Dirk Nowitzki?

  [Baseline]
  Dirk Nowitzki is of German nationality.

  [RAG]
  SPARQL:
    PREFIX nba: <http://nba-kg.org/ontology#>
    PREFIX res: <http://nba-kg.org/resource/>
    SELECT ?nat WHERE { res:Dirk_Nowitzki nba:nationality ?nat . }LIMIT 10
  Answer: German
    [Raw results]
  German

Your question:
```

Figure 10: Interactive RAG session: Embiid nationality, Durant's team, Nowitzki nationality

6 Critical Reflection

6.1 KB Quality

NER extracts entity mentions but no relations. The initial KB contains only `rdf:type`, `rdfs:label`, and `prov:wasDerivedFrom` triples; there are no `playsFor` or `wonAward` facts from the crawled text. SPARQL expansion partially fills this gap but only for the 33 aligned entities. A dedicated relation extraction step would significantly improve KB richness.

Wikidata's `P54` stores full career history, so bridging it directly gives outdated facts (LeBron James linked to Miami Heat rather than the Lakers). We used hardcoded current-season facts as a workaround, but a proper fix would require querying Wikidata for the most recent property values.

6.2 Alignment and Reasoning Limitations

With only 33 manual records, most KB entities have no external link. Scaling would require automatic tools like OpenTapioca or GENRE. SWRL rules are precise but can only derive facts already implicit in the KB. KGE models can predict missing links from structural patterns, but the relation-sparse KB limits the signal available, which explains why the best MRR (0.2562) is well below values seen on dense benchmark graphs like FB15k or WN18RR.

6.3 RAG Limitations

Two failure modes appeared in testing. First, the LLM summariser sometimes rejects retrieved facts that contradict its training data, as seen in Q1. Second, multi-hop queries are unreliable because the small model tends to generate oversimplified SPARQL. The self-repair loop handles syntax errors but cannot fix semantically incorrect queries. Improvements would include relation extraction for a richer KB, a larger LLM (7B+) for more reliable generation, and broader entity alignment.